AI Travel Planner

Multi-agent trip planning with human-in-the-loop review: design report

Stack Python 3.12 · LangGraph 1.2 · LangChain 1.4 · FastAPI · Google Gemini · Exa search · OpenStreetMap · Open-Meteo

Repository github.com/mugil0095/ai-travel-planner **Date** October 2026

1. Summary

A user submits a destination, travel dates, a budget range, interests and the number of travellers. A **Research Agent** gathers live destination intelligence, an **Itinerary Planner Agent** turns it into a timed day-by-day plan, and a **LangGraph orchestrator** pauses the workflow so a human can approve the draft, reject it with feedback, or modify specific days. Only an approved plan becomes final. Every step is checkpointed to SQLite, so a plan waiting for review survives server restarts, and a failed step can be retried without repeating earlier work. The system is served by FastAPI.

2 + 4	**3**	**44**	**E2E**
agents + tools (Exa, Open-Meteo, 2 × OpenStreetMap)	review actions: approve, reject (routed), modify (scoped)	offline tests + 4 live API smoke tests	full live run incl. crash recovery and retry

2. Architecture

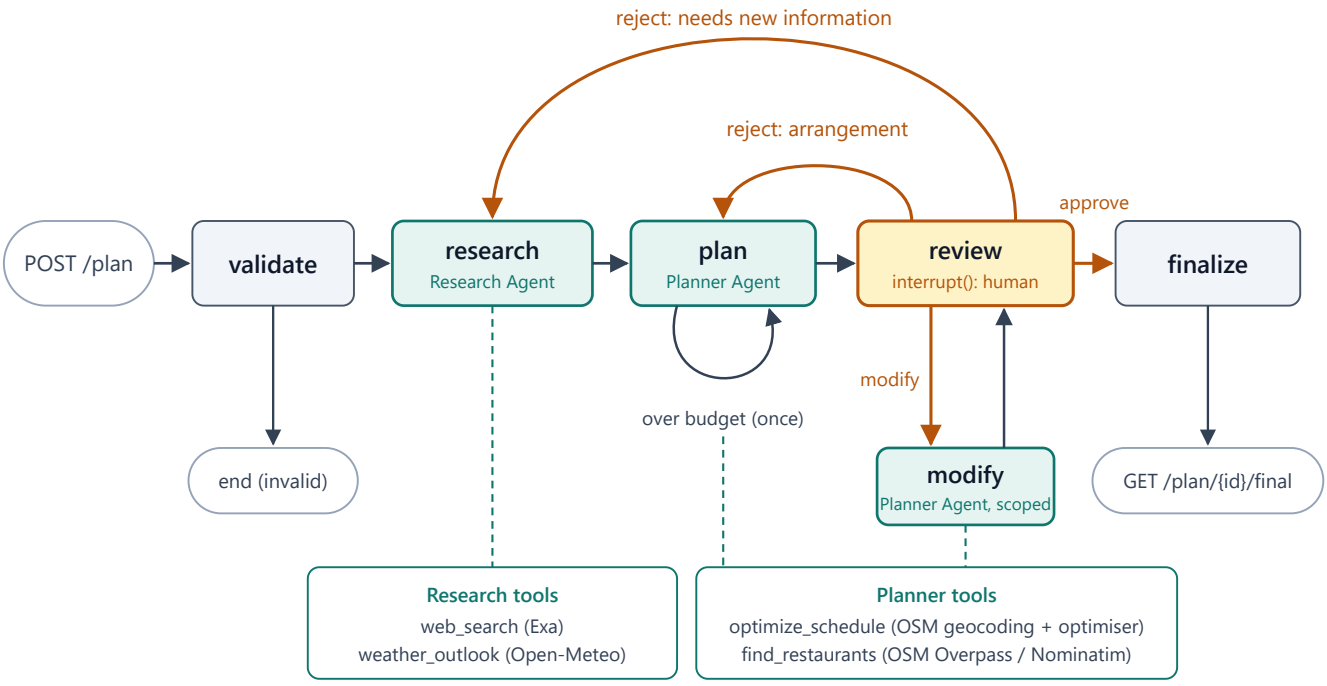


Figure 1. The orchestration graph. Teal = agent nodes, amber = human-in-the-loop routes.

Component	Responsibility
Orchestrator `app/graph.py`	LangGraph `StateGraph`. Validates the request, routes between agents, sends an over-budget draft back to the planner once, pauses at `review` with `interrupt()`, resumes with `Command(resume=…)` and assembles the final plan.
Research Agent `app/agents.py`	Tool-calling agent that returns a typed `ResearchReport`: attractions with durations, prices and opening hours, neighbourhoods, local tips, safety, seasonal notes, typical costs and sources.
Itinerary Planner Agent `app/agents.py`	Tool-calling agent that returns a typed `Itinerary`: lodging, and one entry per day with timed activities, travel minutes and real restaurants. Also applies scoped modifications.
Feedback router	One small LLM call that decides whether a rejection needs new research or only re-planning. The reviewer can override it.
Runner `app/runner.py`	Runs graph executions as background tasks; derives each plan's status from its checkpoint; resumes interrupted runs on startup.
API `app/main.py`	FastAPI endpoints, validation, status codes, server-sent events.

3. Agents and tools

Both agents are LangChain `create_agent` loops (small LangGraph ReAct graphs) that end with a **Pydantic object** (`ToolStrategy`), so the orchestrator never parses free text. Each agent has a model fallback chain and hard per-tool call limits (section 7).

Tool	Agent	How it works
`web_search` mandatory	Research	Exa `/search` with highlights. Returns title, URL, date and the relevant passages for the top 5 results.
`weather_outlook`	Research	Open-Meteo, no key. Trips within the 16-day window get a **real forecast**. Later trips get **last year's observed weather on the same dates**, a better guide than a monthly average.
`optimize_schedule`	Planner	A scheduling optimiser. It geocodes candidate places on OpenStreetMap (inside the city's bounding box first), groups them into days with k-means, moves stops out of over-full days into the nearest day with room, then orders each day to respect **opening hours** and minimise travel. Days with up to 6 stops are searched exhaustively; larger days use nearest neighbour + 2-opt. The output is a timed schedule with walk/transit/drive legs, plus the places that did not fit or could not be found.
`find_restaurants`	Planner	Real restaurants near a place from OpenStreetMap (Overpass radius query), nearest first, with cuisine, hours, address and website, plus cuisine and vegetarian filters. Public Overpass servers are often overloaded, so the tool retries with backoff, tries a mirror, then falls back to Nominatim.

4. Human-in-the-loop and persistence

The `review` node calls `interrupt()`. LangGraph checkpoints the state and the run stops with the plan in `awaiting_review`. `POST /plan/{id}/review` resumes the same thread (`thread_id = plan_id`) with the

reviewer's decision, and the node returns a `Command(goto=…)`:

- **Approve** → `finalize`. This step is deterministic (no LLM), so the final plan is exactly the approved draft plus context.
- **Reject with feedback** → `research` or `plan`. The reviewer can choose with `"revise"`; otherwise the LLM router decides, falling back to the planner if that call fails. The agent receives the previous draft and the feedback.
- **Modify** `[{day, change}]` → `modify`. The planner applies only the requested changes, then the code **restores every day that was not named**, plus the lodging and texts. A change without a day (for example the hotel) is plan-wide.

Runs use `durability="sync"`: each checkpoint is on disk before the next node starts. A plan awaiting review therefore survives restarts. A failed node keeps its error and can be resumed with `POST /retry`, and runs cut off by a shutdown are resumed on startup. Status is derived from the checkpoint, so there is no separate status table to drift out of sync:

Status	Meaning	Allowed actions
`queued`, `processing`	accepted; the next checkpoint is not written yet	—
`researching`, `planning`, `modifying`, `finalizing`	that node is running	—
`awaiting_review`	paused at the interrupt with a draft	approve, reject, modify (approve only after 5 rounds)
`completed`	approved; `/final` is available	—
`failed`, `stalled`	a step raised (error included), or the process stopped mid-run	retry
`invalid`	the orchestrator rejected the request	—

Budget. Agents estimate per-item costs. Totals are computed in code: lodging × nights + (activities + meals + local transport) × travellers. A draft over `budget_max` goes back to the planner once before the human sees it, and every draft carries a breakdown with an `over`, `within` or `under` status.

5. API

Method	Endpoint	Purpose	Responses
POST	`/plan`	Submit a travel request; starts the workflow	`202` + plan_id; `422` invalid input
GET	`/plan/{id}`	Status, draft, research, reviews, progress events	`200`; `404`
POST	`/plan/{id}/review`	Approve, reject (+ feedback, optional target) or modify (+ changes)	`202`; `404`; `409` not awaiting review or limit reached; `422`
GET	`/plan/{id}/final`	The approved plan	`200`; `404`; `409` before approval

GET	`/plan/{id}/events`	Server-sent events, one per status change	`200` event stream
POST	`/plan/{id}/retry`	Resume a failed or stalled run from its checkpoint	`202` ; `409` not failed

A plan takes one to three minutes, so POSTs return `202` immediately, work runs in the background, and clients poll or listen on `/events`. Input validation covers reversed or past dates, trips over 14 days, a minimum budget above the maximum, empty interests, a bad currency code, and unknown days in a modification.

6. Live run

One real session through the API: Lisbon, 20–22 November 2026, 2 travellers, 900–1600 EUR, interests food, history and viewpoints, “relaxed pace; one of us is vegetarian”.

Step	What happened
Research	6 Exa searches + 1 weather call → 10 attractions, 9 sources, typical costs, safety and seasonal notes.
Draft 1	Optimiser called once, restaurant lookup 5 times. Day 1 Graça/Alfama/Baixa/Chiado, day 2 Belém, day 3 Bairro Alto and Cais do Sodré. **897 EUR**, flagged `under` (minimum 900).
Reject	“Too much time on public transport. Keep each day in one area and add a fado evening.” The router sent it to the **research** agent: “introduces a new cultural interest and event type requiring new destination options”.
Draft 2	Fado houses found by research (Fado ao Carmo, Parreirinha de Alfama, Mesa de Frades) added. **1417 EUR**, within budget.
Modify	Day 2: “Replace the afternoon with a sunset at a viewpoint and a vegetarian dinner nearby.” Draft 3 changed only day 2 (Miradouro de Santa Luzia at 16:30); days 1 and 3 and the lodging were **byte-identical**. **1221 EUR**.
Approve	The final plan equals draft 3, with the review history `reject→research, modify, approve` .

Draft 1, day 1 (optimiser output, as returned)

Time	Activity	Travel	Cost pp
09:43–10:43	Miradouro da Senhora do Monte	13 min	free
10:53–12:53	Castelo de São Jorge	10 min	17 EUR
13:00–14:00	Miradouro de Santa Luzia	7 min	free
14:05–15:05	Sé de Lisboa	5 min	7 EUR
15:12–15:57	Arco da Rua Augusta	7 min	5 EUR
16:06–17:36	Museu Arqueológico do Carmo	9 min	7 EUR

Meals: Pastelaria Bénard (breakfast), Casa da Tia Helena (lunch), Mezzogiorno (dinner): real OpenStreetMap restaurants near each area. Drafts 2–3 were produced by `gemini-3.5-flash-lite` after the daily free-tier quota for the full Flash models ran out. It over-applied the fado request to every night, and its “vegetarian dinner” is a general Portuguese tasca.

7. Testing and what live testing found

Suite	Tests	Covers
`test_graph.py`	10	Every HITL path, routing both ways, scoped and plan-wide modify, budget loop, invalid input, restart persistence, failure then retry
`test_api.py`	13	Status codes, full reject/modify/approve flow over HTTP, revision limit, retry, SSE, restart
`test_tools.py`	18	Clustering, opening hours, overflow, travel modes, weather source choice, Overpass fallback, Exa request shape, budget maths
`test_runner.py`, `test_agent_wiring.py`	3	Status during in-flight steps; the real agents (middleware, tools, structured output) inside the real graph with a scripted model
`test_live.py` (opt-in)	4	Real Open-Meteo, Nominatim, Overpass and Exa calls

The offline tests fake the LLM agents. The live run exposed five problems they could not, all now fixed:

1. **Status race.** The status briefly read `invalid` mid-run: between a step's writes and its checkpoint, the snapshot shows no next node. A running plan is now never reported as terminal (regression test added).
2. **Overloaded upstreams.** Gemini returned `503`, and the free tier allows 20 requests per model per day; Overpass returned `502`. Fixes: a model fallback chain (`ModelFallbackMiddleware`), and the retry / mirror / Nominatim chain for restaurants.
3. **Runaway tool use.** One model verified every attraction, one search at a time, until it hit the recursion limit. Fixes: per-tool caps (`ToolCallLimitMiddleware`) and parallel searches in one turn.
4. **LangGraph edge case.** Agents compiled with `checkpointer=False` crash under a parent run with `durability="sync"`. The agents now inherit the checkpointer, and a wiring test runs them inside the real graph.
5. **Recovery verified.** The server was killed mid-research and resumed the plan on restart. A plan that failed on quota errors resumed via `/retry` without repeating research.

8. Design decisions and tradeoffs

1. **Asynchronous API (202 + polling or SSE).** Runs take minutes on the free tier, so holding HTTP requests open is fragile. The cost is that clients must poll or subscribe.
2. **The checkpoint is the single source of truth.** Status, drafts, reviews and errors are all read from LangGraph state, so retry and restart recovery come almost for free. The cost is that listing plans scans checkpoints; at scale you would add an index table.
3. **Typed agent outputs.** Pydantic schemas via structured output: no free-text parsing, and invalid output is retried inside the agent.
4. **Deterministic code wherever correctness matters:** budget totals, the modify guard, the final plan and status derivation. LLMs are used only for judgement: research, planning, routing.
5. **Precise rejection routing** with a reviewer override and a safe fallback, so feedback reaches the agent that can act on it.
6. **Free, keyless data for 3 of the 4 tools** keeps the project runnable with two API keys. The cost is accuracy: travel times are straight-line × 1.3 at a speed per mode, not real routing.
7. **Single process.** asyncio background tasks plus a per-process lock are correct for one worker; multiple workers need a queue and a shared lock (section 10).

9. Improvements with more time

- Real routing (OSRM or Google Routes) and parsing of OSM opening-hours strings in the optimiser.
- A currency tool, for live rates instead of LLM conversion.
- Stream agent tool calls into the SSE feed, not only node-level progress.
- An evaluation set of trip requests scored for budget, opening hours, day count and real places, to compare prompts and models.
- Direct JSON edits of a draft, alongside natural-language modifications.
- A small web UI for the review step.

10. Production concerns

- **Scale:** run graphs from a queue (Celery, Arq or LangGraph Platform); Postgres checkpointer; database locks so two workers cannot resume one thread.
- **Reliability:** idempotency keys, per-node timeouts, circuit breakers and caching for external APIs.
- **Cost and abuse:** authentication, per-user quotas, token and cost accounting per plan, and a revision cap (already in place).
- **Observability:** tracing of agent steps, structured logs with `plan_id`, latency and error metrics.
- **Safety:** treat web content as untrusted (prompt injection), keep PII out of logs, respect OSM usage policy or self-host.
- **Data lifecycle:** TTL cleanup and backups for checkpoints.

11. Assumptions

- The budget is the total for the whole group: lodging, food, activities and local transport, not travel to the destination. It is in the request currency; the LLM converts local prices.
- Trips are 1–14 days, start today or later, for 1–20 travellers. Nights = days − 1.
- “Modify” means targeted changes to named days, or plan-wide changes such as the hotel. “Reject” means the draft needs a new pass.
- Prices, hours and events come from web search and the LLM's reading of it: estimates to verify before booking.